

PROGRAMACIÓN I

UTN INSPT - Comisión 1.605 - Ciclo 2026

Guía de Ejercicios — Unidad 4: Funciones

Temas:

definición, prototipo y llamada
Pasaje por valor y por referencia
cohesión funcional — reutilización

Docentes:

Prof. Ing. Alejandro Jorge Behringer
Ay. Prof. Juan Carlos Capia

Regla de la catedra:

Una función que calcula NO imprime. Una función que imprime NO calcula. El main solo coordina.

Ejercicio 1 Factorial iterativo ★★☆☆☆

Tema: Bucles — acumulación multiplicativa — funciones

1. Implementar la función factorial(n) que calcule n! de forma iterativa.
2. Llamar a la función desde main con los valores: 0, 1, 5 y 10, mostrando el resultado de cada llamada.
3. Usar la firma obligatoria indicada abajo.

Prototipos:

```
long factorial(int n);

/* Llamadas en main */
printf("factorial(0) = %ld\n", factorial(0));
printf("factorial(1) = %ld\n", factorial(1));
printf("factorial(5) = %ld\n", factorial(5));
printf("factorial(10) = %ld\n", factorial(10));
```

Llamada	Resultado esperado
factorial(0)	1
factorial(1)	1
factorial(5)	120
factorial(10)	3628800

Restricciones:

- factorial() calcula y retorna — no imprime.
- Usar long como tipo de retorno para soportar valores grandes.
- El caso n = 0 debe retornar 1 (por definición matemática).
- No usar recursividad.

Ejercicio 2 Combinatoria $C(n, k)$ ★★★★★**Tema:** Composición de funciones — reutilización de factorial

1. Calcular de cuántas formas se pueden seleccionar k personas de un grupo de n candidatos.
2. Implementar la función `combinatoria(n, k)` que use la función `factorial()` del Ejercicio 1.
3. Solicitar n y k al usuario. Validar que k sea menor o igual a n y que ambos sean positivos.
4. Mostrar el resultado con el formato: $C(n, k) = n! / (k! \times (n-k)!) = \text{resultado}$.

Formula matemática:

$$C(n, k) = n! / (k! \times (n-k)!)$$

Ejemplo con $n=10, k=3$:

$$C(10, 3) = 10! / (3! \times 7!) = 120$$

Prototipos:

```
long factorial(int n);  
long combinatoria(int n, int k);
```

Restricciones:

- `combinatoria()` llama a `factorial()` tres veces — no re implementa el cálculo.
- `factorial()` no se modifica respecto del Ejercicio 1.
- `combinatoria()` calcula y retorna — no imprime.
- La presentación del resultado va en `main`.

Ejercicio 3 Numero primo ★★☆☆☆

Tema: Funciones booleanas — divisibilidad

1. Solicitar un numero entero positivo mayor que 1. Validar el ingreso.
2. Implementar la función **esPrimo(n)** que retorne 1 si n es primo, 0 si no lo es.
3. Mostrar si el numero ingresado es primo o no.
4. Listar todos los números primos entre 2 y 100 reutilizando **esPrimo()**.

Prototipos:

```
int esPrimo(int n);
```

Restricciones:

- **esPrimo()** evalúa y retorna un entero (1 o 0) — no imprime.
- La presentación del resultado va en main.
- El listado de primos entre 2 y 100 reutiliza **esPrimo()** — no repite la lógica.
- Un numero primo es divisible solo por 1 y por si mismo.

Ejercicio 4 Mayor de tres números ★★☆☆☆

Tema: Funciones de comparación — composición

1. Solicitar tres numeros reales al usuario. Validar que sean distintos entre si.
2. Implementar la **funcion mayorDeDos(a, b)** que retorne el mayor entre dos valores.
3. Usar **mayorDeDos()** dos veces para encontrar el mayor de los tres — sin if adicional en main para comparar los tres números.
4. Mostrar el resultado e indicar si el mayor es positivo, negativo o cero (función aparte).

Prototipos:

```
float mayorDeDos(float a, float b);  
int determinarSigno(float n); /* retorna: 1=positivo, -1=negativo, 0=cero */
```

Restricciones:

- mayorDeDos() solo compara dos valores — nada más.
- **determinarSigno()** solo evalúa el signo — no compara magnitudes.
- Ninguna de las dos funciones imprime.
- **Prohibido usar if en main para comparar los tres números directamente.**

Ejercicio 5 Numero perfecto ★★★★★**Tema:** Divisores — acumulación — función booleana

1. Solicitar un numero entero positivo mayor que 1. Validar el ingreso.
2. Implementar **sumaDivisoresPropios(n)** que retorne la suma de todos los divisores de n excepto n mismo.
3. Implementar **esPerfecto(n)** que retorne 1 ó true si n es igual a su suma de divisores propios, 0 ó false si no.
4. Mostrar el resultado indicando si el número es perfecto.
5. Permitir consultas múltiples hasta que el usuario ingrese 0 para salir.

Prototipos:

```
long sumaDivisoresPropios(int n);  
int esPerfecto(int n);
```

Restricciones:

- **sumaDivisoresPropios()** solo suma — no decide si es perfecto.
- **esPerfecto()** llama a **sumaDivisoresPropios()** — no recalcula divisores.
- Ninguna de las dos funciones imprime.
- Un divisor propio de n es todo i tal que $n \% i == 0$, con $1 \leq i < n$.

Ejemplos: 6 es perfecto ($1+2+3 = 6$). 28 es perfecto ($1+2+4+7+14 = 28$). 12 no lo es (suma = 16).

Ejercicio 6 análisis de dígitos ★★★★★**Tema:** extracción de dígitos — reutilización de funciones

1. Solicitar un numero entero positivo de al menos 2 dígitos. Validar.
2. Implementar **cantidadDigitos(n)** que retorne cuantos dígitos tiene el número.
3. Implementar **digitoMaximo(n)** que retorne el digito más grande del número.
4. Implementar **digitoMinimo(n)** que retorne el digito más pequeño del número.
5. Implementar **sumarDigitos(n)** que retorne la suma de todos los dígitos.
6. Calcular el promedio de los dígitos usando **sumarDigitos()** y **cantidadDigitos()**. Mostrar con dos decimales.

Prototipos:

```
int cantidadDigitos(int n);  
int digitoMaximo(int n);  
int digitoMinimo(int n);  
int sumarDigitos(int n);
```

Restricciones:

- Cada función extrae los dígitos con su propio ciclo — no comparten estado.
- sumarDigitos() ya implementada — reutilizarla para el promedio, no reescribirla.
- técnica de extracción: $\text{digito} = n \% 10$; $n = n / 10$;
- Ninguna de las funciones imprime.

Ejemplo con 3842: $\text{max} = 8$, $\text{min} = 2$, $\text{suma} = 17$, $\text{cantidad} = 4$, $\text{promedio} = 4.25$

Ejercicio 7 Invertir un número - capicúa ★★★★★

Tema: construcción del inverso — comparación numérica

1. Solicitar un numero entero positivo mayor que 9 (al menos 2 dígitos). Validar.
2. Implementar **invertirNumero(n)** que construya y retorne el numero con sus dígitos invertidos.
3. Implementar **esCapicua(n)** que retorne 1 si $n == \text{invertirNumero}(n)$, 0 si no.
4. Mostrar el numero original, el invertido y si es o no capicúa.
5. Buscar e imprimir todos los capicúas entre 10 y 999 usando las funciones anteriores.

Prototipos:

```
int invertirNumero(int n);  
int esCapicua(int n);
```

```
/* técnica de construcción del inverso */  
/* Con n = 1234 se extraen: 4, 3, 2, 1 -> se construye 4321 */  
inverso = inverso * 10 + (n % 10);  
n       = n / 10;
```

Restricciones:

- invertirNumero() solo construye el inverso — no decide si es capicúa.
- esCapicua() solo llama a invertirNumero() y compara — no extrae dígitos directamente.
- El ciclo de búsqueda de capicúas entre 10 y 999 va en main.
- Ninguna de las dos funciones imprime.

Ejercicio 8 MCD y MCM — Algoritmo de Euclides ★★★★★**Tema:** Algoritmo de Euclides — composición de funciones

1. Solicitar dos enteros positivos A y B. Validar que ambos sean mayores que cero.
2. Implementar `mcd(a, b)` usando el algoritmo de Euclides iterativo (con `while`).
3. Implementar `mcm(a, b)` que llame a `mcd()` — no que Re implemente el algoritmo.
4. Mostrar: MCD, MCM, y si los números son coprimos (`MCD == 1`).
5. Permitir múltiples consultas. Al finalizar mostrar cuantos pares coprimos se ingresaron.

Prototipos:

```
int mcd(int a, int b);  
int mcm(int a, int b);
```

```
/* Algoritmo de Euclides iterativo */  
/* MCD(48, 18) -> MCD(18,12) -> MCD(12,6) -> MCD(6,0) = 6 */  
/* El ciclo termina cuando b llega a 0 — el MCD queda en a */  
/* MCM(a, b) = (a * b) / MCD(a, b) */
```

Ejercicio 9 clasificación numérica — abundante, perfecto o deficiente ★★★★★**Tema:** reutilización de sumaDivisoresPropios — funciones de clasificación

1. Solicitar un entero positivo mayor que 1. Validar.
2. Reutilizar la función sumaDivisoresPropios() del Ejercicio 5.
3. Implementar tres funciones: esAbundante(n), esPerfecto(n), esDeficiente(n) — cada una llama a sumaDivisoresPropios().
4. Clasificar el número ingresado y mostrar el resultado.
5. Mostrar cuantos números abundantes hay entre 1 y 100.

Prototipos:

```
long sumaDivisoresPropios(int n); /* reutilizada del Ejercicio 5 */
int  esAbundante(int n);        /* suma > n */
int  esPerfecto(int n);         /* suma == n */
int  esDeficiente(int n);       /* suma < n */
```

Restricciones:

- Las tres funciones de clasificación solo llaman a sumaDivisoresPropios() y comparan.
- sumaDivisoresPropios() no se modifica ni se duplica — se reutiliza exactamente como esta.
- Ninguna de las funciones imprime.

Ejemplos: 12 es abundante (suma=16 > 12). 8 es deficiente (suma=7 < 8). 6 es perfecto.

Ejercicio 10 Juego: Adivina el numero — disen modular ★★★★★**Tema:** Integrador — azar, validación, ciclos, funciones de un propósito

1. El programa genera un número secreto aleatorio entre 1 y 100. El jugador tiene hasta 10 intentos.
2. Implementar generarSecreto(min, max) que retorne un entero aleatorio en el rango [min, max].
3. Implementar validarIntento(intento, min, max) que retorne 1 si el intento está en rango, 0 si no.
4. Implementar comparar(intento, secreto) que retorne: 1 si intento > secreto, -1 si intento < secreto, 0 si son iguales.
5. En cada intento, informar si el secreto es MAYOR o MENOR al ingresado y cuantos intentos quedan.
6. Al finalizar mostrar: número secreto, intentos usados y una calificación según tabla abajo.
7. Permitir jugar varias partidas. Al final mostrar cuantas partidas gano el jugador.

Prototipos:

```
int generarSecreto(int min, int max);  
int validarIntento(int intento, int min, int max);  
int comparar(int intento, int secreto);
```

Intentos usados	Calificacion
1 a 3	Extraordinario
4 a 6	Muy bien
7 a 9	Justo a tiempo
10 sin acertar	Mejor suerte la próxima

Restricciones:

- generarSecreto() solo genera — no valida ni muestra nada.
- comparar() solo compara y retorna un entero (1, -1 o 0) — no muestra mensajes.
- Los mensajes de pista y calificación se muestran en main.
- El ciclo del juego termina por DOS condiciones: acerto O agoto los intentos.
- Fórmula: $\text{rand()} \% (\text{max} - \text{min} + 1) + \text{min}$